

```

#include <Wire.h>
#include <SPI.h>

#include <Adafruit_GFX.h>
#include <Adafruit_ST7789.h>
#include <Adafruit_AHTX0.h>
#include <Adafruit_BMP280.h>
#include <Adafruit_Sensor.h>

#define TFT_CS    5
#define TFT_DC    2
#define TFT_RST   4
#define TFT_MOSI  23
#define TFT_SCLK  18

#define I2C_SDA    21
#define I2C_SCL    22

#define LED_GREEN  25
#define LED_YELLOW 26
#define LED_RED    27

#define BUZZER_PIN 32

const float WARNING_TEMP = 30.0;
const float DANGER_TEMP  = 38.0;

Adafruit_ST7789 tft(
  TFT_CS,
  TFT_DC,
  TFT_RST
);

Adafruit_AHTX0 aht;
Adafruit_BMP280 bmp;

bool ahtWorking = false;
bool bmpWorking = false;

#define COLOR_BLACK      0x0000
#define COLOR_PANEL      0x1082
#define COLOR_PANEL_2    0x18C3
#define COLOR_BORDER     0x3186

#define COLOR_WHITE      0xFFFF
#define COLOR_GREY       0x8410
#define COLOR_LIGHT_GREY 0xC618
#define COLOR_DARK_GREY  0x4208

#define COLOR_BLUE       0x05FF
#define COLOR_CYAN       0x07FF

#define COLOR_GREEN      0x07E0
#define COLOR_DARK_GREEN 0x0320

#define COLOR_YELLOW     0xFFE0
#define COLOR_ORANGE     0xFD20

#define COLOR_RED        0xF800
#define COLOR_DARK_RED   0x6000

enum TemperatureState {
  SAFE_STATE,
  WARNING_STATE,
  DANGER_STATE
};

TemperatureState currentState = SAFE_STATE;

float temperatureC = 0.0;
float humidityPercent = 0.0;
float pressureHpa = 0.0;

unsigned long lastSensorRead = 0;
unsigned long lastBuzzerChange = 0;
unsigned long lastStatusBlink = 0;

```

```
const unsigned long SENSOR_INTERVAL = 1000;
```

```
bool buzzerOn = false;  
bool statusBlinkOn = true;
```

```
void printCentered(  
    String text,  
    int centerX,  
    int y,  
    int textSize,  
    uint16_t color  
) {  
    int16_t x1;  
    int16_t y1;  
    uint16_t width;  
    uint16_t height;  
  
    tft.setTextWrap(false);  
    tft.setTextSize(textSize);  
    tft.setTextColor(color);  
  
    tft.getTextBounds(  
        text,  
        0,  
        y,  
        &x1,  
        &y1,  
        &width,  
        &height  
    );  
  
    tft.setCursor(  
        centerX - width / 2,  
        y  
    );  
  
    tft.print(text);  
}
```

```
void drawTemperatureIcon(  
    int x,  
    int y,  
    uint16_t color  
) {  
    tft.drawRoundRect(  
        x + 6,  
        y,  
        10,  
        27,  
        5,  
        color  
    );  
  
    tft.fillCircle(  
        x + 11,  
        y + 29,  
        7,  
        color  
    );  
  
    tft.fillRect(  
        x + 9,  
        y + 9,  
        5,  
        20,  
        color  
    );  
}
```

```
void drawHumidityIcon(  
    int x,  
    int y,  
    uint16_t color  
) {  
    tft.fillTriangle(  
        x + 11,  
        y + 29,  
        x + 11,  
        y + 29,  
        x + 11,  
        y + 29,  
        color  
    );  
}
```

```

        x + 11,
        y,
        x + 2,
        y + 18,
        x + 20,
        y + 18,
        color
    );

    tft.fillCircle(
        x + 11,
        y + 18,
        9,
        color
    );

    tft.fillCircle(
        x + 8,
        y + 16,
        2,
        COLOR_PANEL
    );
}

void drawPressureIcon(
    int x,
    int y,
    uint16_t color
) {
    tft.drawCircle(
        x + 13,
        y + 13,
        12,
        color
    );

    tft.drawCircle(
        x + 13,
        y + 13,
        9,
        color
    );

    tft.drawLine(
        x + 13,
        y + 13,
        x + 19,
        y + 7,
        color
    );

    tft.fillCircle(
        x + 13,
        y + 13,
        2,
        color
    );
}

void drawBootScreen() {
    tft.fillScreen(COLOR_BLACK);

    tft.drawRoundRect(
        18,
        35,
        204,
        215,
        18,
        COLOR_BLUE
    );

    tft.drawRoundRect(
        22,
        39,
        196,
        207,

```

```

    15,
    COLOR_PANEL_2
);

printCentered(
    "OMAR",
    120,
    70,
    3,
    COLOR_CYAN
);

printCentered(
    "ENVIRONMENT",
    120,
    115,
    2,
    COLOR_WHITE
);

printCentered(
    "MONITOR",
    120,
    150,
    3,
    COLOR_GREEN
);

printCentered(
    "Temperature Safety System",
    120,
    210,
    1,
    COLOR_GREY
);

tft.drawRoundRect(
    35,
    270,
    170,
    12,
    5,
    COLOR_BORDER
);

for (int width = 0; width <= 166; width += 5) {
    tft.fillRoundRect(
        37,
        272,
        width,
        8,
        4,
        COLOR_BLUE
    );

    delay(25);
}

delay(600);
}

void drawDashboard() {
    tft.fillScreen(COLOR_BLACK);

    tft.fillRoundRect(
        8,
        8,
        224,
        44,
        10,
        COLOR_PANEL
    );

    tft.drawRoundRect(
        8,
        8,

```

```

    224,
    44,
    10,
    COLOR_BLUE
);

tft.setTextColor(COLOR_CYAN);
tft.setTextSize(2);
tft.setCursor(17, 16);
tft.print("ENV MONITOR");

tft.setTextColor(COLOR_GREY);
tft.setTextSize(1);
tft.setCursor(18, 38);
tft.print("LIVE ENVIRONMENT DATA");

tft.fillCircle(
    212,
    29,
    5,
    COLOR_GREEN
);

tft.fillRoundRect(
    8,
    61,
    224,
    84,
    12,
    COLOR_PANEL
);

tft.drawRoundRect(
    8,
    61,
    224,
    84,
    12,
    COLOR_BORDER
);

drawTemperatureIcon(
    18,
    82,
    COLOR_ORANGE
);

tft.setTextColor(COLOR_GREY);
tft.setTextSize(1);
tft.setCursor(52, 72);
tft.print("TEMPERATURE");

tft.fillRoundRect(
    8,
    154,
    108,
    67,
    12,
    COLOR_PANEL
);

tft.drawRoundRect(
    8,
    154,
    108,
    67,
    12,
    COLOR_BORDER
);

drawHumidityIcon(
    17,
    174,
    COLOR_CYAN
);

```

```

tft.setTextColor(COLOR_GREY);
tft.setTextSize(1);
tft.setCursor(17, 162);
tft.print("HUMIDITY");

tft.fillRect(
    124,
    154,
    108,
    67,
    12,
    COLOR_PANEL
);

tft.drawRoundRect(
    124,
    154,
    108,
    67,
    12,
    COLOR_BORDER
);

drawPressureIcon(
    132,
    174,
    COLOR_BLUE
);

tft.setTextColor(COLOR_GREY);
tft.setTextSize(1);
tft.setCursor(134, 162);
tft.print("PRESSURE");

tft.fillRect(
    8,
    230,
    224,
    64,
    12,
    COLOR_PANEL
);

tft.drawRoundRect(
    8,
    230,
    224,
    64,
    12,
    COLOR_BORDER
);

tft.setTextColor(COLOR_GREY);
tft.setTextSize(1);
tft.setCursor(18, 240);
tft.print("TEMPERATURE STATUS");

tft.setTextColor(COLOR_DARK_GREY);
tft.setTextSize(1);
tft.setCursor(22, 306);
tft.print("AHT20 + BMP280 SAFETY SYSTEM");
}

TemperatureState calculateState(
    float temperature
) {
    if (temperature >= DANGER_TEMP) {
        return DANGER_STATE;
    }

    if (temperature >= WARNING_TEMP) {
        return WARNING_STATE;
    }

    return SAFE_STATE;
}

```

```

uint16_t getStateColor(
    TemperatureState state
) {
    if (state == DANGER_STATE) {
        return COLOR_RED;
    }

    if (state == WARNING_STATE) {
        return COLOR_YELLOW;
    }

    return COLOR_GREEN;
}

String getStateName(
    TemperatureState state
) {
    if (state == DANGER_STATE) {
        return "DANGER";
    }

    if (state == WARNING_STATE) {
        return "WARNING";
    }

    return "SAFE";
}

void updateLEDs(
    TemperatureState state
) {
    digitalWrite(
        LED_GREEN,
        LOW
    );

    digitalWrite(
        LED_YELLOW,
        LOW
    );

    digitalWrite(
        LED_RED,
        LOW
    );

    if (state == SAFE_STATE) {
        digitalWrite(
            LED_GREEN,
            HIGH
        );
    }

    if (state == WARNING_STATE) {
        digitalWrite(
            LED_YELLOW,
            HIGH
        );
    }

    if (state == DANGER_STATE) {
        digitalWrite(
            LED_RED,
            HIGH
        );
    }
}

void updateBuzzer() {
    unsigned long now = millis();

    if (currentState == SAFE_STATE) {
        noTone(BUZZER_PIN);
        buzzerOn = false;
        return;
    }
}

```

```

}

unsigned long silentDuration;
unsigned long soundDuration;
int frequency;

if (currentState == WARNING_STATE) {
    float progress =
        (temperatureC - WARNING_TEMP) /
        (DANGER_TEMP - WARNING_TEMP);

    progress = constrain(
        progress,
        0.0f,
        1.0f
    );

    silentDuration =
        1300 -
        (unsigned long)(progress * 900);

    soundDuration = 130;

    frequency =
        1400 +
        (int)(progress * 500);
} else {
    float progress =
        (temperatureC - DANGER_TEMP) /
        12.0f;

    progress = constrain(
        progress,
        0.0f,
        1.0f
    );

    silentDuration =
        350 -
        (unsigned long)(progress * 260);

    soundDuration =
        170 +
        (unsigned long)(progress * 80);

    frequency =
        2200 +
        (int)(progress * 1300);
}

if (!buzzerOn) {
    if (
        now - lastBuzzerChange >=
        silentDuration
    ) {
        tone(
            BUZZER_PIN,
            frequency
        );

        buzzerOn = true;
        lastBuzzerChange = now;
    }
} else {
    if (
        now - lastBuzzerChange >=
        soundDuration
    ) {
        noTone(BUZZER_PIN);

        buzzerOn = false;
        lastBuzzerChange = now;
    }
}
}
}

```

```

void drawTemperatureBar(
    float temperature,
    uint16_t color
) {
    int x = 52;
    int y = 127;
    int width = 162;
    int height = 8;

    tft.fillRect(
        x,
        y,
        width,
        height,
        4,
        COLOR_DARK_GREY
    );

    float limitedTemperature =
        constrain(
            temperature,
            15.0f,
            50.0f
        );

    int filledWidth = map(
        (int)(limitedTemperature * 10),
        150,
        500,
        0,
        width
    );

    if (filledWidth > 0) {
        tft.fillRect(
            x,
            y,
            filledWidth,
            height,
            4,
            color
        );
    }
}

```

```

void drawTemperatureValue() {
    uint16_t color =
        getStateColor(currentState);

    tft.fillRect(
        51,
        84,
        170,
        39,
        COLOR_PANEL
    );

    tft.setTextColor(color);
    tft.setTextSize(4);
    tft.setCursor(54, 87);
    tft.print(temperatureC, 1);

    tft.drawCircle(
        176,
        91,
        3,
        color
    );

    tft.setTextSize(2);
    tft.setCursor(183, 96);
    tft.print("C");

    drawTemperatureBar(
        temperatureC,
        color
    );
}

```

```

    );
}

void drawHumidityValue() {
    tft.fillRect(
        43,
        178,
        66,
        34,
        COLOR_PANEL
    );

    tft.setTextColor(COLOR_CYAN);
    tft.setTextSize(2);
    tft.setCursor(47, 184);

    tft.print(
        humidityPercent,
        0
    );

    tft.print("%");
}

void drawPressureValue() {
    tft.fillRect(
        160,
        177,
        65,
        37,
        COLOR_PANEL
    );

    if (bmpWorking) {
        tft.setTextColor(COLOR_ORANGE);
        tft.setTextSize(2);
        tft.setCursor(161, 181);

        tft.print(
            pressureHpa,
            0
        );

        tft.setTextColor(COLOR_GREY);
        tft.setTextSize(1);
        tft.setCursor(182, 203);
        tft.print("hPa");
    } else {
        tft.setTextColor(COLOR_RED);
        tft.setTextSize(1);
        tft.setCursor(164, 188);
        tft.print("NO BMP");
    }
}

void drawStatus() {
    uint16_t color =
        getStateColor(currentState);

    String statusText =
        getStateName(currentState);

    tft.fillRect(
        15,
        253,
        210,
        35,
        COLOR_PANEL
    );

    tft.fillRoundRect(
        18,
        253,
        204,
        31,
        9,

```

```

        color
    );

    uint16_t textColor =
        COLOR_WHITE;

    if (
        currentState ==
        WARNING_STATE
    ) {
        textColor = COLOR_BLACK;
    }

    printCentered(
        statusText,
        120,
        261,
        2,
        textColor
    );
}

void updateScreenValues() {
    drawTemperatureValue();
    drawHumidityValue();
    drawPressureValue();
    drawStatus();
}

void drawSensorError(
    String sensorName
) {
    tft.fillScreen(COLOR_BLACK);

    tft.drawRoundRect(
        15,
        75,
        210,
        155,
        15,
        COLOR_RED
    );

    printCentered(
        "SENSOR ERROR",
        120,
        102,
        2,
        COLOR_RED
    );

    printCentered(
        sensorName,
        120,
        145,
        2,
        COLOR_WHITE
    );

    printCentered(
        "CHECK CONNECTIONS",
        120,
        185,
        1,
        COLOR_YELLOW
    );

    printCentered(
        "SDA 21 / SCL 22",
        120,
        205,
        1,
        COLOR_GREY
    );
}

```

```

bool startBMP280() {
    if (
        bmp.begin(
            0x76
        )
    ) {
        Serial.println(
            "BMP280 found at 0x76"
        );

        return true;
    }

    if (
        bmp.begin(
            0x77
        )
    ) {
        Serial.println(
            "BMP280 found at 0x77"
        );

        return true;
    }

    return false;
}

bool readSensors() {
    sensors_event_t humidityEvent;
    sensors_event_t temperatureEvent;

    if (!ahtWorking) {
        return false;
    }

    aht.getEvent(
        &humidityEvent,
        &temperatureEvent
    );

    float newTemperature =
        temperatureEvent.temperature;

    float newHumidity =
        humidityEvent.relative_humidity;

    if (
        isnan(newTemperature) ||
        isnan(newHumidity)
    ) {
        return false;
    }

    temperatureC =
        newTemperature;

    humidityPercent =
        newHumidity;

    if (bmpWorking) {
        float pressurePa =
            bmp.readPressure();

        if (!isnan(pressurePa)) {
            pressureHpa =
                pressurePa / 100.0f;
        }
    }

    currentState =
        calculateState(
            temperatureC
        );

    updateLEDs(

```

```

        currentState
    );

    return true;
}

void printSensorData() {
    Serial.print("Temperature: ");
    Serial.print(temperatureC, 1);
    Serial.print(" C");

    Serial.print(" | Humidity: ");
    Serial.print(humidityPercent, 1);
    Serial.print(" %");

    Serial.print(" | Pressure: ");

    if (bmpWorking) {
        Serial.print(pressureHpa, 1);
        Serial.print(" hPa");
    } else {
        Serial.print("Not available");
    }

    Serial.print(" | Status: ");
    Serial.println(
        getStateName(
            currentState
        )
    );
}

void setup() {
    Serial.begin(115200);
    delay(500);

    pinMode(
        LED_GREEN,
        OUTPUT
    );

    pinMode(
        LED_YELLOW,
        OUTPUT
    );

    pinMode(
        LED_RED,
        OUTPUT
    );

    digitalWrite(
        LED_GREEN,
        LOW
    );

    digitalWrite(
        LED_YELLOW,
        LOW
    );

    digitalWrite(
        LED_RED,
        LOW
    );

    pinMode(
        BUZZER_PIN,
        OUTPUT
    );

    noTone(
        BUZZER_PIN
    );

    SPI.begin(

```

```

    TFT_SCLK,
    -1,
    TFT_MOSI,
    TFT_CS
);

tft.init(
    240,
    320
);

tft.setRotation(2);

tft.invertDisplay(true);

tft.fillScreen(
    COLOR_BLACK
);

drawBootScreen();

Wire.begin(
    I2C_SDA,
    I2C_SCL
);

Wire.setClock(
    400000
);

ahtWorking =
    aht.begin(
        &Wire
    );

if (!ahtWorking) {
    Serial.println(
        "AHT20 not found"
    );

    drawSensorError(
        "AHT20 NOT FOUND"
    );

    digitalWrite(
        LED_RED,
        HIGH
    );

    while (true) {
        tone(
            BUZZER_PIN,
            1500
        );

        delay(130);

        noTone(
            BUZZER_PIN
        );

        delay(1000);
    }
}

Serial.println(
    "AHT20 connected"
);

bmpWorking =
    startBMP280();

if (bmpWorking) {
    bmp.setSampling(
        Adafruit_BMP280::MODE_NORMAL,
        Adafruit_BMP280::SAMPLING_X2,

```

```

Adafruit_BMP280::SAMPLING_X16,
Adafruit_BMP280::FILTER_X16,
Adafruit_BMP280::STANDBY_MS_500
);

Serial.println(
  "BMP280 connected"
);
} else {
  Serial.println(
    "BMP280 not found"
  );
}

drawDashboard();

delay(300);

if (readSensors()) {
  updateScreenValues();
  printSensorData();
}

lastSensorRead =
  millis();
}

void loop() {
  unsigned long now =
    millis();

  if (
    now - lastSensorRead >=
    SENSOR_INTERVAL
  ) {
    lastSensorRead = now;

    if (readSensors()) {
      updateScreenValues();
      printSensorData();
    }
  }

  updateBuzzer();
}

```